

A Level Computer Science (9618)

Topical Past Paper Worksheet

Topic 4: Processor Fundamentals

Subtopic 4.2: Assembly Language

Concept 4.2.3: Tracing Assembly Programs

Total Questions: 14

Mark Schemes begin on Page 28

Question 1 | Source: 9618_w25_qp_12 (Q3.b)

- (b) The following table shows part of the instruction set for a processor. The processor has two registers: the ACC and an IX.

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127		

The current contents of the main memory and the index register are shown.

Address	Instruction
98	8
99	16
100	3
101	98
102	32
IX	2

Write the contents of the ACC after each instruction is executed.

Instruction	Value in ACC
LDM #98	
LDI 101	
LDX 100	

[3]

- 8 (a) The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system

ACC denotes Accumulator
 <address> can be an absolute or a symbolic address
 # denotes a denary number, e.g. #123
 B denotes a binary number, e.g. B01001010
 & denotes a hexadecimal number, e.g. &4A

The current contents of memory are:

address	Instruction
80	10
81	8
82	80
83	81
...	
200	LDD 81
201	INC ACC
202	STO 83
203	LDI 82
204	CMP 83
205	JPE 209
206	LDD 83
207	ADD #10
208	JMP 210
209	DEC ACC
210	STO 81
211	END

Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address			
		80	81	82	83
		10	8	80	81

[4]

Question 3 | Source: 9618_s25_qp_12 (Q7.a.i)

- 7 (a) The following table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDM	#n	Immediate addressing. Load the number n to ACC
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC)
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JMP	<address>	Jump to the given address
END		Return control to the operating system

ACC denotes Accumulator
 <address> can be an absolute or a symbolic address
 # denotes a denary number, e.g. #123
 B denotes a binary number, e.g. B01001010
 & denotes a hexadecimal number, e.g. &4A

The current contents of memory are:

Address Instruction

10	12
11	11
12	10
13	22
14	22
...	
100	LDD 10
101	ADD 12
102	STO 11
103	CMP 14
104	JPE 107
105	INC ACC
106	JMP 101
107	STO 10
108	END

(i) Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address				
		10	11	12	13	14
		12	11	10	22	22

[3]

Question 4 | Source: 9618_s25_qp_13 (Q5.a.i)

- 5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system

ACC denotes Accumulator
 <address> can be an absolute or a symbolic address
 # denotes a denary number, e.g. #123
 B denotes a binary number, e.g. B01001010
 & denotes a hexadecimal number, e.g. &4A

- (a) The current contents of memory are:

Address	Instruction
50	47
51	48
52	49
53	50
...	
500	LDM #50
501	DEC ACC
502	CMP 51
503	JPE 505
504	JMP 501
505	STO 50
506	SUB #10
507	STO 51
508	END

(i) Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address			
		50	51	52	53
		47	48	49	50

[3]

Question 5 | Source: 9618_s24_qp_11 (Q4.a)

- 4 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
19	24
20	2
21	1
22	3
23	5
24	4
25	22

The current contents of the ACC and IX are shown:

ACC	12
IX	1

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDD 20 ADD #2	
2	LDX 22	
3	LDI 25 INC ACC SUB 22	
4	LDD 19 LDM #5 LDM #25	

[4]

Question 6 | Source: 9618_s24_qp_12 (Q5.a)

- 5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
10	1
11	3
12	5
13	11
14	10
15	16
16	12

The current contents of the ACC and IX are shown:

ACC	10
IX	0

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDI 15 SUB #1	
2	LDD 14 ADD 11	
3	LDM #11 ADD #3 SUB 16	
4	LDR #2 LDX 14 ADD #2	

[4]

Question 7 | Source: 9618_s24_qp_13 (Q3.a)

- 3 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
48	51
49	6
50	48
51	50
52	49
53	50
54	6

The current contents of the ACC and IX are shown:

ACC	2
IX	50

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDM #50 INC ACC SUB #1	
2	LDI 51 ADD 52	
3	LDR #2 LDX 50 DEC ACC	
4	LDD 52 SUB 54 INC ACC	

[4]

Question 8 | Source: 9618_w23_qp_12 (Q9.b)

- 9 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	#n	Add the denary number n to the ACC
JMP	<address>	Jump to the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123		

- Trace the program currently in memory using the trace table when the input is '1'.

[4]

Question 9 | Source: 9618_w22_qp_11 (Q6.a.i)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (i) Trace the program currently in memory using the trace table.

ASCII value	Character
49	1
50	2
51	3
52	4
	
65	A
66	B
67	C
68	D

Question 10 | Source: 9618_w22_qp_11 (Q6.a.ii)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(a) The current contents of main memory and selected values from the ASCII character set are given.

(ii) The following instructions are repeated for your reference.

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address

State the purpose of this part of an assembly language program.

```
LDD 100
STO 165
LDD 101
STO 100
LDD 165
STO 101
```

.....

.....

[1]

Question 11 | Source: 9618_w22_qp_12 (Q7.a)

- 7 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (a) Trace the program currently in memory using the trace table, stopping when line 90 is executed for a second time.

Address	Instruction	Instruction address	ACC	IX	Memory address						
					100	101	102	103	110	111	112
75	LDR #0				0	0	112	4	1	4	0
76	LDX 110										
77	CMI 102										
78	JPE 91										
79	CMP 103										
80	JPN 84										
81	ADD 101										
82	STO 101										
83	JMP 86										
84	INC ACC										
85	STO 101										
86	LDD 100										
87	INC ACC										
88	STO 100										
89	INC IX										
90	JMP 76										
91	END										
...	↗										
100	0										
101	0										
102	112										
103	4										
...	↗										
110	1										
111	4										
112	0										

[5]

Question 12 | Source: 9618_w22_qp_13 (Q6.a.i)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Moves the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
SUB	#n	Subtract the denary number n from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	#n	Compare the contents of ACC with number n
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (i) Trace the program currently in memory using the trace table.

ASCII value	Character
49	1
50	2
51	3
52	4
	
65	A
66	B
67	C
68	D

[4]

3 A processor has one general purpose register, the Accumulator (ACC), and several special purpose registers.

The following table shows part of the instruction set for a processor. The processor has a 16-bit general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Move the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMF	<address>	Compare the contents of ACC with the contents of <address>
JPF	<address>	Following a compare instruction, jump to <address> if the compare was True
JPM	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end

<address> can be an absolute address or a symbolic address
 # denotes a denary number, e.g. #123

The current contents of the main memory and selected values from the ASCII character set are shown.

Address	Instruction	ASCII code table (selected codes only)	
200	LDD 365	65	A
201	CMP 366	66	B
202	JPE 209	67	C
203	INC ACC	68	D
204	STO 365		
205	MOV IX		
206	LDX 365		
207	OUT		
208	JMP 200		
209	END		
—			
365	1		
366	3		
367	65		
368	66		
IX	0		

Complete the trace table for the program currently in main memory.

[illegible]

Question 14 | Source: 9618_s21_qp_12 (Q4.a)

- 4 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system

denotes a denary number, e.g. #123

The current contents of the main memory and selected values from the ASCII character set are:

Address Instruction

70	IN
71	CMP 100
72	JPE 80
73	CMP 101
74	JPE 76
75	JMP 80
76	LDD 102
77	INC ACC
78	STO 102
79	JMP 70
80	LDD 102
81	DEC ACC
82	STO 102
83	JMP 70
...	
100	68
101	65
102	100

ASCII code table (selected codes only)

ASCII code	Character
65	A
66	B
67	C
68	D

MARK SCHEMES

Mark Scheme for Question 1 | Source: 9618_w25_ms_12 (Q3.b)

3(b)	1 mark for each correct value, max 3 marks	3								
<table><tr><th>Instruction</th><th>Value in ACC</th></tr><tr><td>LDM # 98</td><td>98</td></tr><tr><td>LDI 101</td><td>8</td></tr><tr><td>LDX 100</td><td>32</td></tr></table>			Instruction	Value in ACC	LDM # 98	98	LDI 101	8	LDX 100	32
Instruction	Value in ACC									
LDM # 98	98									
LDI 101	8									
LDX 100	32									

Mark Scheme for Question 2 | Source: 9618_s25_ms_11 (Q8.a)

8(a)

1 mark for each shaded section

4

Instruction address	ACC	Memory address			
		80	81	82	83
		10	8	80	81
200	8				
201	9				
202					9
203	10				
204					
206	9				
207	19				
208					
210			19		

7(a)(i)

1 mark for each shaded section

3

Instruction address	ACC	Memory address				
		10	11	12	13	14
		12	11	10	22	22
100	12					
101	22					
102			22			
103						
104						
107		22				

Mark Scheme for Question 4 | Source: 9618_s25_ms_13 (Q5.a.i)

5(a)(i)

1 mark for each shaded section

3

Instruction address	ACC	Memory address			
		50	51	52	53
		47	48	49	50
500	50				
501	49				
502					
504					
501	48				
502					
503					
505		48			
506	38				
507			38		

Mark Scheme for Question 5 | Source: 9618_s24_ms_11 (Q4.a)

4(a)	1 mark for each correct answer:	4
------	---------------------------------	---

Mark Scheme for Question 6 | Source: 9618_s24_ms_12 (Q5.a)

5(a)

1 mark for each correct answer:

4

Program Number	Code	ACC Content
1	LDI 15 SUB #1	11
2	LDD 14 ADD 11	13
3	LDM #11 ADD #3 SUB 16	2
4	LDR #2 LDX 14 ADD #2	14

Mark Scheme for Question 7 | Source: 9618_s24_ms_13 (Q3.a)

3(a)

1 mark for each correct answer:

4

Program Number	Code	ACC Content
1	LDM #50 INC ACC SUB #1	50
2	LDI 51 ADD 52	97
3	LDR #2 LDX 50 DEC ACC	48
4	LDD 52 SUB 54 INC ACC	44

Mark Scheme for Question 8 | Source: 9618_w23_ms_12 (Q9.b)

9(b)

1 mark for each set of shaded rows

4

Instruction address	ACC	IX	Memory address				Output
			100	101	110	111	
			0	0	51	65	
10		0					
11	49						
12				49			
13	51						
14							
15							
16	49						
17	65						
18		1					
19			65				
20							
13	65						
14							
15							
21							A
22							

6(a)(i)

1 mark for each set of highlighted rows.

4

Instruction address	ACC	IX	Memory address					Output
			100	101	110	111	112	
			0	0	66	65	35	
77		0						
78	66							
79								
80								
81								
82				66				
83	1							
84								
85			1					
86		1						
87	65							
88								
89								
81	66							
82								
83	1							
84	2							
85			2					
86		2						
87	35							
88								
89								
90	2							
91	50							
92								2
93								

Mark Scheme for Question 10 | Source: 9618_w22_ms_11 (Q6.a.ii)

6(a)(ii)	swaps the contents of memory address <u>100</u> and <u>101</u>	1
----------	--	----------

7(a) 1 mark for each set of highlighted rows.

5

Instruction address	ACC	IX	Memory address						
			100	101	102	103	110	111	112
			0	0	112	4	1	4	0
75		0							
76	1								
77									
78									
79									
80									
84	2								
85				2					
86	0								
87	1								
88			1						
89		1							
90									
76	4								
77									
78									
79									
80									
81	6								
82				6					
83									
86	1								
87	2								
88			2						
89		2							
90									

Mark Scheme for Question 12 | Source: 9618_w22_ms_13 (Q6.a.i)

6(a)(i)

1 mark for each set of highlighted rows:

4

Instruction address	ACC	IX	Memory address					Output
			100	101	110	111	112	
			1	0	97	98	97	
75		0						
76	1							
77								
78								
79	97							
80	65							
81								
82								
83	1							
84								
85				1				
86								
87	2							
88			2					
89		1						
90								
76								
77								
78								
91	1							
92	49							
93								1
94								

3(b)

1 mark for each shaded set of values

6

Instruction address	ACC	Memory address				IX	Output
		365	366	367	368		
		1	3	65	66	0	
200	1						
201							
202							
203	2						
204		2					
205						2	
206	65						
207							A
208							
200	2						
201							
202							
203	3						
204		3					
205						3	
206	66						
207							B
208							
200	3						
201							
202							
209							

4(a)

1 mark for each shaded section / bullet point

4

- Load 65 into ACC
- Load 100 into ACC, increment and store in 102
- Load 68 into ACC
- Load 101 into ACC, decrement and store in 102

Instruction address	ACC	Memory address		
		100	101	102
		68	65	100
70	65			
71				
72				
73				
74				
76	100			
77	101			
78				101
79				
70	68			
71				
72				
80	101			
81	100			
82				100
83				
(70)				